Contents

Nexus AI Investment Research System — Final Version (March 2026) **2**
Architecture & Data Quality Framework . . . 2
1. System Overview . . . 2
1.1 Purpose . . . 2
1.2 Value Proposition . . . 2
1.3 Four-Agent Architecture . . . 3
2. Data Quality Framework . . . 4
2.1 Architecture . . . 4
2.2 Layer 1 — Deterministic Checks . . . 5
2.3 Layer 2 — Cross-Source Validation . . . 7
2.4 Layer 3 — LLM-Assisted Audit . . . 8
2.5 Core Metrics Prioritisation . . . 8
2.6 Calendarization Methodology . . . 9
2.7 Scoring System . . . 10
2.8 Report Format . . . 10
2.9 Prompt Engineering & Responsible AI Practices . . . 11
2.10 Data Modelling, Processing, Integration & Analytics Expertise . . . 12
3. Current System Status . . . 14
3.1 Data Coverage . . . 14
3.2 FMP Starter Plan Realities . . . 14
3.3 Automated Update Cadence . . . 15
3.4 Future-Proofing . . . 15
3.5 Observability & Workflow Transformation . . . 15
4. Technology Stack . . . 16
5. Compliance & Institutional Standards . . . 17
5.1 Future-Proof Architecture: Path to LangGraph Migration . . . 17
5.2 Product Roadmap & Stakeholder Collaboration . . . 19
6. GitHub Strategy . . . 20
6.1 Repository Setup . . . 20
6.2 Making the Project Interview-Ready . . . 20
6.3 Protecting Proprietary Information . . . 21
7. Development Log . . . 21
Week 1 — Foundation (Feb 22–23, 2026) . . . 21
Week 1 — Analysis Engine (Feb 24–28, 2026) . . . 21
Week 2 — Institutional Quality (Mar 1–2, 2026) . . . 22
8. Total Cost Summary . . . 22
8.1 Infrastructure Costs . . . 22
8.2 Cumulative Costs (Feb 22 – Mar 2, 2026) . . . 22
8.3 Cost Context . . . 23
9. Agentic Framework Comparison — Interview Preparation . . . 23
9.1 Framework Overview . . . 23
9.2 How Nexus Would Look in LangGraph . . . 25
9.3 Sample Interview Answers . . . 25
10. Full Replication Guide . . . 26
Prerequisites . . . 26
Step 1: Project Scaffolding and Core Architecture . . . 27

Step 2: Data Pipeline and Macro Indicators . . . 27
Step 3: Telegram Bot and Agent 2 Analysis . . . 27
Step 4: FMP Data Collection Framework . . . 28
Step 5: Universe Manager and Batch Orchestration . . . 28
Step 6: Web Search and Research Data Collection . . . 29
Step 7: DCF and Terminal Growth Modelling . . . 29
Step 8: Vector RAG System . . . 30
Step 9: Agent 4 — Research Polishing and Daily Valuation . . . 30
Step 10: Calendarization Engine . . . 31
Step 11: Data Quality Audit Framework . . . 31
Step 12: Future-Proofing and Documentation . . . 32

Nexus AI Investment Research System — Final Version (March 2026)

Architecture & Data Quality Framework

---

1. System Overview

1.1 Purpose

Nexus is an institutional-grade AI investment research platform designed to automate the collection, validation, analysis, and delivery of equity research across the S&P 500 universe. The system ingests financial data from multiple sources, normalises it to a standardised calendar-quarter framework, applies a rigorous three-layer data quality audit, and delivers actionable investment analysis through both a web interface and a Telegram-based research assistant.

The platform is built to meet the standards expected by institutional investors: full citation tracking on every data point, deterministic auditability, cross-source validation, and conservative analytical methodology grounded in discounted cash flow models, multi-factor analysis, and consensus-driven estimates.

1.2 Value Proposition

- **Full S&P 500 coverage**: 504 companies tracked with automated weekly full refreshes and daily incremental updates
- **Institutional citation standards**: Every financial data point carries source name, URL, publication date, and verbatim quote text
- **Automated data quality assurance**: 35 deterministic and cross-source checks plus LLM-assisted anomaly classification, running daily
- **Calendar-quarter normalisation**: Day-weighted proration eliminates fiscal-year-end distortion for all 504 companies, including non-standard fiscal calendars (4-5-4, non-December FYE)
- **Professional research delivery**: AI-generated analysis governed by institutional tone enforcement and competitive benchmarking against WSJ, FT, Bloomberg, and Reuters standards

1.3 Four-Agent Architecture

Nexus operates on a specialised four-agent architecture, where each agent has a clearly defined scope and responsibility boundary.

Agent 1 — Data Collector Agent 1 is responsible for all external data acquisition and internal data storage. No other agent writes to the database.

- **Universe management**: Tracks S&P 500 composition via iShares IVV ETF holdings, detecting additions, removals, and delistings
- **Five sub-collectors per ticker**: Consensus estimates (Revenue/EPS, annual periods), financial statements (income/balance sheet/cash flow, last 12 quarters), capital structure (WACC, beta, enterprise value, net debt), earnings transcripts (prepared remarks + Q&A with AI-parsed sentiment), and revenue segmentation (product and geographic breakdown)
- **Research data collector**: Web search-based collection of growth prospects, market share, company guidance, institutional ownership, management changes, and pricing trends — stored with full citations
- **Rate-limited orchestration**: Batch processing in groups of 5 with `Promise.allSettled` for fault isolation; 1.5-second delays between web searches
- **AI-enhanced functions**: Automated coverage scanning for new opportunities, financial data validation, field reconciliation, and restatement interpretation

Agent 2 — Analyst Agent 2 is the research and analysis engine. It reads from the database but never writes raw data — only analytical outputs.

- **Contextual awareness**: Maintains a 90-day rolling window of news, newsletter data, and prior analyses to ensure consistency and avoid repetition
- **Nexus Score**: Proprietary 0–100 composite rating based on five pillars — Factor Position, Financial Momentum, Valuation, Earnings Revisions, and Sentiment & Positioning
- **Trade Setup synthesis**: Multi-factor weighted analysis combining DCF fair value (25%), style/momentum (25%), earnings revisions (20%), industry dynamics (15%), and catalysts/risks (15%)
- **DCF modelling**: Live Excel-compatible models with segment-weighted terminal growth rates, exponential fade functions, and sensitivity analysis across 4–9% terminal growth range
- **Adaptive response format**: Quick Take, Full Analysis, and Conversational modes with automatic factor/style chart generation
- **Consistency enforcement**: Maintains prior directional stances unless data changes; distinguishes new information from already-priced-in developments

Agent 3 — Scheduler Agent 3 manages all automated operations and ensures the system remains current without manual intervention.

Schedule	Operation	Description
Sunday 10:00 PM UTC	Full data update	All 504 tickers refreshed across all 5 sub-collectors
Weekdays 00:30 UTC	Incremental update	Recent earnings reporters + random sample of 35 stale tickers

Schedule	Operation	Description
Weekdays 00:00 UTC	Earnings calendar	Updates upcoming earnings dates, auto-refreshes recent reporters
Sunday 8:00 PM UTC	Universe refresh	Synchronises S&P 500 composition against IVV ETF holdings
Daily 10:00 PM UTC	Macro data update	Rates, yields, inflation, GDP, employment, sentiment indicators
Daily 11:00 PM UTC	Morning briefing	AI-generated market briefing from RSS news feeds
Daily 6:00 PM UTC	Agent 4 valuation & audit	System valuation, data quality audit, Telegram broadcast
Tue/Sat 5:00 PM UTC	Research polishing	Competitive analysis and style benchmarking

All scheduled operations broadcast progress and completion summaries to linked Telegram users.

Agent 4 — Research & Polishing Intelligence Agent 4 provides autonomous quality assurance, competitive intelligence, and institutional tone enforcement.

- **Daily valuation**: System-wide audit producing a fair value estimate with low/base/high USD range, scored across architecture, data quality, analytical quality, and compliance readiness
- **Data quality audit**: Three-layer audit framework (detailed in Section 2) running daily after valuation
- **Research polishing**: Analyses competitor AI tools (Morningstar, FactSet, Bloomberg Terminal), TradFi commentary standards, and investment forum theses to generate actionable style/format/tone suggestions
- **Institutional guardrails**: Enforces conservative, professional tone — "Never adopt hype or retail-style language"
- **Cooldown-protected**: 3-second delays between web searches, maximum 10 searches per polishing run

---

2. Data Quality Framework

2.1 Architecture

The Nexus Data Quality Framework is a three-layer audit system designed to catch data errors at progressively higher levels of sophistication.

```
Layer 1: Deterministic Checks (30 checks)
    Automated, rule-based validation of freshness, units, completeness,
    sanity, fiscal accuracy, and citation integrity.

Layer 2: Cross-Source Validation (5 checks)
```

```
    Comparison across independent data sources to detect inconsistencies
    that pass individual-source validation.

Layer 3: LLM-Assisted Audit (up to 5 tickers/day)
    AI-driven anomaly classification for the worst-scoring tickers,
    distinguishing genuine data errors from expected business events.
```

2.2 Layer 1 — Deterministic Checks

30 automated checks across six categories, each producing a pass/fail result with severity classification (critical, high, medium, low).

Category A: Source Document Currency & Freshness (6 checks)

ID	Check	Threshold	Severity
1	Financial data staleness	Latest period > 120 days old	High
2	Collection recency	Ticker not updated in 14 days	Medium
3	Source fetch lag	Record fetched > 90 days after period close	Low
4	Post-earnings gap	Reported earnings but no matching period in DB	High
5	Research data age	Research data > 30 days old	Low
6	Transcript freshness	Transcript coverage as % of S&P 500	Low

Category B: Number Units Validation (6 checks)

ID	Check	Threshold	Severity
7	Revenue magnitude jump	QoQ change > 10x	Critical
8	Net income magnitude jump	QoQ change > 10x	High
9	Revenue-to-NI ratio	Net margin > 80% or < -200%	Medium
10	Revenue vs sector median	Revenue > 100x or < 0.01x sector median	High
11	Shares outstanding range	Shares < 1M or > 50B	Medium
12	Market cap vs price x shares	Market cap inconsistent with price x shares	Medium

Category C: Core Metrics Completeness (3 checks)

ID	Check	Metrics Required	Severity
13	Income statement	9 metrics (revenue, grossProfit, ebitda, operatingIncome, netIncome, eps, weightedAverageShsOut, R&D expenses, SG&A expenses)	High
14	Balance sheet	7 metrics (totalAssets, totalDebt, totalStockholdersEquity, cashAndCashEquivalents, totalCurrentAssets, totalCurrentLiabilities, goodwill)	High
15	Cash flow	5 metrics (operatingCashFlow, capitalExpenditure, freeCashFlow, commonStockRepurchased, stockBasedCompensation)	High

Category D: Numerical Sanity & Outliers (7 checks)

ID	Check	Threshold	Severity
16	Gross margin range	GM < -10% or > 98%	Medium
17	EPS cross-check	Computed EPS vs reported EPS diverges > 10%	High
18	P/E ratio range	P/E outside -500 to 500	Medium
19	Beta range	Beta < 0.1 or > 4.0	Medium
20	D/E ratio range	D/E < -1 or > 20 (excl. Financials sector)	Medium
21	Enterprise value	Negative EV for tickers with market cap > $1B	Medium
22	EBITDA margin	EBITDA margin < -50% or > 80%	Medium

Category E: Fiscal Period Accuracy & Calendarization (5 checks)

ID	Check	Description	Severity
23	Duplicate records	Identical ticker/metric/period entries	High
24	Period ordering	Chronological sequence errors across quarters	Medium
25	FYE alignment	Q4 period-end month vs fiscal_year_end_month metadata	High
26	TTM consistency	Negative or zero trailing twelve-month revenue	Medium
27	Missing quarters	Fewer than 6 quarters of data in last 2 years	Medium

Category F: Citations & Data Integrity (3 checks)

ID	Check	Description	Severity
28	Citation completeness	Percentage of records with source_name and source_url	High
29	Segment weight sum	Segment revenue sum vs total reported revenue (50–200% tolerance)	Medium
30	Missing paired data	Tickers with financials but no estimates, or vice versa	Medium

2.3 Layer 2 — Cross-Source Validation

5 checks that compare data across independent tables and sources to detect inconsistencies invisible to single-source validation.

ID	Check	What it compares	Tolerance
31	Estimate vs actual divergence	Calendarized quarterly estimates vs calendarized actuals for most recent quarters	$> 50\%$ gap

ID	Check	What it compares	Tolerance
32	Calendarization coverage gap	Raw financial/estimate tickers vs successfully calendarized tickers	> 20 ticker gap
33	TTM revenue vs annual estimate	Calendarized trailing twelve-month revenue vs latest annual consensus estimate	> 200% gap
34	Capital structure vs implied shares	Stored shares outstanding vs net income / EPS implied shares	> 50% gap
35	Seasonality weight consistency	Sum of calendarized quarterly estimates vs source annual consensus total	> 1% drift

2.4 Layer 3 — LLM-Assisted Audit

For the bottom-scoring tickers that trigger multiple Layer 1/2 flags, the system engages GPT-5-mini to perform contextual anomaly classification.

Selection criteria: Score < 90/100 with 2 or more failed checks. Maximum 5 tickers reviewed per day.

Context provided to the LLM: - Most recent 20 financial data points (metric name, value, period, statement type, citation) - Last 3 capital structure snapshots (market cap, shares, beta, WACC, EV, D/E) - Last 10 consensus estimates (period, metric, estimate, actual, analyst count) - Full details of each failed check (name, severity, description)

Classification output: - **Actionable**: Genuine data error requiring correction (wrong units, stale data, missing records) - **Monitor**: Ambiguous situation to watch in the next reporting cycle - **Benign**: Expected anomaly due to M&A, restructuring, seasonal business, IPO, or spin-off

Daily rate limit: Enforced at the database level — if a Layer 3 report already exists for the current date, subsequent audit runs skip LLM calls entirely.

2.5 Core Metrics Prioritisation

The system tracks 28 key financial fields across three statement types, selected for their relevance to institutional equity analysis:

Income Statement (9 fields): revenue, grossProfit, ebitda, operatingIncome, netIncome, eps (diluted), weightedAverageShsOut, researchAndDevelopmentExpenses, sellingGeneralAndAdministrativeExpenses

Balance Sheet (7 fields): totalAssets, totalDebt, totalStockholdersEquity, cashAndCashEquivalents, totalCurrentAssets, totalCurrentLiabilities, goodwill

Cash Flow (7 fields): operatingCashFlow, capitalExpenditure, freeCashFlow, commonStockRepurchased, stockBasedCompensation, acquisitionsNet, dividendsPaid

Valuation & Structure (5 fields): marketCap, enterpriseValue, sharesOutstanding, beta, wacc

2.6 Calendarization Methodology

Nexus implements day-weighted proration to normalise all financial data to standard calendar quarters (January–March, April–June, July–September, October–December), eliminating distortion from non-December fiscal year ends.

The Problem Of 504 tracked companies, 99 have non-December fiscal year ends. Companies like Costco (FYE August, 4-5-4 retail calendar), Apple (FYE September), and Microsoft (FYE June) report quarters that straddle calendar boundaries. Without calendarization, period-over-period comparisons and aggregate statistics are distorted.

The Solution The `computeCalendarWeights(fqStartStr, fqEndStr)` function accepts actual fiscal quarter start and end dates (not month-end approximations) and computes day-weighted overlap with each calendar quarter.

For quarter-aligned companies (FYE in March, June, September, or December): `blend_method = 'direct'` with 1:1 mapping and `weight_coverage = 1.0`.

For non-aligned companies: `blend_method = 'prorated'` with day-weighted splits. For example, a fiscal quarter ending January 31 would be split approximately 97% to Q1 and 3% to Q4 of the prior year, based on actual day counts.

For balance sheet items: Point-in-time semantics — the most recent value within each calendar quarter is used, not a prorated blend.

Accuracy improvement: Using actual consecutive period_end dates (previous period_end + 1 day) rather than month-end approximation eliminated approximately 7 percentage points of error for 4-5-4 fiscal calendar companies.

Coverage

Metric	Count
Total calendarized financial records	176,045
Quarter-aligned (direct mapping)	151,313
Prorated (day-weighted split)	24,732
Tickers covered	504 / 504 (100%)

Estimates Calendarization Annual consensus estimates are split into calendar quarters using seasonal weights derived from the most recent four quarters of actual financial data. This produces company-specific quarterly estimates that reflect historical seasonality patterns.

All calendarized estimates carry an explicit `basis_warning` flagging the fundamental mismatch between non-GAAP consensus estimates and GAAP-reported actuals.

Metric	Count
Calendarized estimate records	40,208

Metric	Count
Tickers covered	504 / 504 (100%)
Source method	Annual seasonal split (100%)

2.7 Scoring System

Overall score formula: `(checks_passed / checks_total) x 90 + citation_bonus + FYE_bonus + ticker_average_bonus`

- Citation bonus: +5 if 100% citation completeness
- FYE bonus: +3 if all FYE alignment checks pass
- Ticker average bonus: +5 if mean ticker score exceeds 80/100

Per-ticker scoring: Each ticker starts at 100 and loses points per failed check based on severity: - Critical: -15 points - High: -8 points - Medium: -4 points - Low: -2 points

Severity deduction thresholds: The number of affected tickers determines whether a check passes or fails. Thresholds are calibrated to the S&P 500 universe size (e.g., > 3 tickers with revenue magnitude jumps triggers a critical flag).

2.8 Report Format

The daily data quality report follows a structured format designed for rapid review:

```
NEXUS DATA QUALITY REPORT
Date: 2026-03-02

OVERALL SCORE: 67/100

SUMMARY:
  Layer 1 (Deterministic): 19/30 passed (63%)
  Layer 2 (Cross-Source):  3/5 passed (60%)
  Layer 3 (LLM-Assisted): 5 tickers reviewed, 3 actionable
  Total: 22/35 passed | Failed: 13

CRITICAL FLAGS (action required):
  ! #7 Revenue magnitude jump
    3 tickers have revenue changes > 10x Q-over-Q

HIGH PRIORITY WARNINGS:
  * #1 Financial data staleness
    13 tickers have no financial data or latest period > 120 days old

CHECK RESULTS BY CATEGORY:
  A. Source Document Currency & Freshness: 3/6 passed
  B. Number Units Validation: 4/6 passed
  C. Core Metrics Completeness: 3/3 passed
  D. Numerical Sanity & Outliers: 3/7 passed
  E. Fiscal Period Accuracy & Calendarization: 4/5 passed
```

```
  F. Citations & Data Integrity: 2/3 passed
  G. Cross-Source Validation: 3/5 passed

LAYER 2: CROSS-SOURCE VALIDATION:
  [OK] #32 Calendarization coverage gap
       Financials: 504/504 calendarized (gap: 0).
       Estimates: 504/504 calendarized (gap: 0).

LAYER 3: LLM-ASSISTED AUDIT (5 tickers reviewed):
  [!!] AES (actionable)
       Assessment: Revenue jump from missing prior-quarter data
       Root cause: Mis-timestamped fiscal quarter boundaries
       Action: Verify and correct prior-quarter revenue entries
  [??] MRNA (monitor)
       Assessment: Lumpy biotech revenue, legitimate negative EBITDA
       Root cause: Seasonal vaccine delivery timing
       Action: Cross-check with 10-Q filings
  Summary: 3 actionable, 2 monitor, 0 benign
```

2.9 Prompt Engineering & Responsible AI Practices

Structured Prompt Design Every LLM interaction in Nexus follows a disciplined prompt architecture designed for consistency, auditability, and institutional tone compliance:

- **Role definitions**: Each agent receives a precise role boundary in its system prompt. Agent 2 is instructed as "the Cognitive Skills Engine of Nexus — an AI-driven investment research framework built on proprietary S&P 500 factor backtesting data." Agent 4 operates as a "senior fintech valuation analyst with deep expertise in AI-powered financial platforms."
- **Few-shot examples**: Trade setup prompts include structured examples of multi-factor synthesis output, showing the expected format for DCF integration, factor positioning, and risk/catalyst analysis — ensuring consistent output structure across hundreds of analysis requests.
- **Tone guardrails**: All research-facing prompts enforce the directive: *"Conservative, professional, institutional tone. Never adopt hype or retail-style language."* Agent 4's polishing pipeline continuously benchmarks output against WSJ, FT, Bloomberg, and Reuters standards.
- **Output format enforcement**: JSON response format (`response_format: { type: "json_object" }`) is used for structured outputs (daily topics, valuation scorecards, quality reports). Free-text outputs use sentence-aware line wrapping for Telegram delivery compatibility.

Conservative Language Enforcement The system enforces conservative analytical language through multiple layers:

- **Directional stance consistency**: Agent 2 maintains prior directional stances unless new data justifies a change, and explicitly distinguishes new information from already-priced-in developments
- **Uncertainty disclosure**: All outputs that compare consensus estimates to reported actuals carry explicit basis warnings noting the fundamental mismatch between non-GAAP consensus and GAAP-reported figures

- **Growth assumption boundaries**: Terminal growth rates in DCF models are capped using exponential fade functions — the system never extrapolates high near-term growth rates indefinitely, applying industry-profile-specific decay (k=0.12 for secular growth, k=0.30 for mature industries)
- **Citation requirements**: Every data-driven claim in research output must reference the underlying data source. The system prompt instructs: *"Cite specific numbers: factor Q1-Q5 spreads, macro indicator values, daily/weekly performance"*

Responsible AI Principles

- **Explicit over implicit**: Missing data is flagged rather than silently substituted. Coverage gaps are surfaced in reports rather than papered over with estimates.
- **Deterministic reproducibility**: Layer 1 and Layer 2 audit checks produce identical results on identical data — no stochastic elements in quality assurance.
- **Human oversight by design**: The system generates research and recommendations but does not execute trades or take automated market actions. All outputs are delivered to human users for review.
- **Rate-limited LLM usage**: Layer 3 audit limits LLM calls to 5 tickers per day, enforced at the database level. This prevents runaway API costs and ensures each LLM review receives adequate context and attention.
- **Bias mitigation**: Factor positioning uses cross-sectional quintile ranking (equal-weighted by construction), avoiding market-cap-weighted biases that favour mega-cap companies in aggregate analysis.

2.10 Data Modelling, Processing, Integration & Analytics Expertise

Data Modelling The PostgreSQL schema follows a normalised relational design across 15+ tables, each serving a distinct analytical purpose:

- **Core entity tables**: `companies` (S&P 500 constituents with sector, industry, FYE month, profile metadata), `company_financials` (per-metric granularity across income/balance/cashflow statements), `consensus_estimates` (Revenue + EPS annual estimates)
- **Derived analytical tables**: `calendarized_financials` (day-weighted proration metadata per record), `calendarized_estimates` (seasonal-split quarterly estimates with basis warnings)
- **Supporting tables**: `capital_structure` (WACC, beta, EV, D/E per ticker), `company_segments` (product + geographic revenue breakdown), `company_transcripts` (prepared remarks + Q&A with management tone), `company_research_data` (web-sourced qualitative intelligence with full citations)
- **Audit and operational tables**: `data_quality_reports`, `system_valuation_reports`, `collection_runs`, `research_polish_reports`

The schema enforces fiscal-to-calendar mapping at the record level — every financial data point carries both its original `period_end` (fiscal quarter boundary) and derived `calendar_quarter` (normalised CQ designation), enabling joins across companies with different fiscal year ends without runtime transformation.

Data Processing & Integration Multi-source data pipelines with fault-tolerant orchestration:

- **Primary source**: FMP API with rate-limited client (300 calls/min, auto-retry on 429 errors) feeding five parallel sub-collectors per ticker

- **Secondary source**: DuckDuckGo + Bing web search with CAGR sentence extraction, market research site prioritisation, and 12-second timeout enforcement
- **Tertiary source**: Earnings transcript ingestion from FMP with Seeking Alpha and BamSEC web fallback, automatic vector embedding on insertion
- **Citation integrity**: Every record carries `source_name`, `source_url`, `published_date`, and `quote_text` — no data enters the system without provenance
- **Batch orchestration**: `Promise.allSettled` groups of 5 tickers for fault isolation — individual ticker failures do not halt batch processing; all runs logged to `collection_runs` with start/end timestamps, ticker counts, and error details

Analytics Capabilities

- **Day-weighted calendarization**: `computeCalendarWeights()` uses actual fiscal quarter start/end dates (not month-end approximations) for day-count overlap with calendar quarters, achieving ~7 percentage points better accuracy for 4-5-4 fiscal calendar companies
- **Trailing twelve-month (TTM) calculations**: `getCalendarizedTTM()` sums last 4 calendar quarters of income statement and cash flow metrics, requiring `weight_coverage >= 0.9` for inclusion — incomplete quarters are explicitly excluded rather than estimated
- **Seasonal estimate splitting**: Annual consensus estimates decomposed to calendar quarters using trailing 4-quarter actual data as seasonal weights, with `basis_warning` flags on every derived record
- **DCF modelling**: Segment-weighted terminal growth rates with exponential fade functions (industry-profile-specific decay constants: k=0.12 for secular growth, k=0.30 for mature industries), producing low/base/high fair value ranges
- **Factor positioning**: Cross-sectional quintile ranking across momentum, value, quality, growth, and yield styles with daily Q1-Q5 spread tracking
- **Automated quality assurance**: 35-check audit framework with per-ticker scoring, severity-weighted deductions, and LLM-assisted anomaly classification

Enterprise Platform Readiness The database schema includes nullable future-proofing columns designed for seamless migration to enterprise data platforms:

Column	Table	Purpose
`reporting_currency` (varchar 3)	companies, company_financials	Multi-currency support (EUR, GBP, JPY)
`usd_fx_rate` (real)	company_financials	Foreign exchange conversion tracking
`accounting_standard` (varchar 10)	companies, company_financials	IFRS vs US GAAP distinction
`reporting_language` (varchar 5)	companies	Multilingual filing support
`primary_exchange` (varchar 10)	companies	Multi-exchange coverage (LSE, TSE, HKEX)

These columns are currently unused (all S&P 500 companies report in USD under US GAAP) but ensure that expanding to international equities requires no schema migration — only data population.

The architecture demonstrates proficiency in enterprise-grade data workflows even on a lightweight stack. The normalised schema, citation-tracked pipelines, and deterministic audit framework translate directly to Snowflake, Databricks, or BigQuery environments. The calendarization engine, scoring system, and batch orchestration patterns are platform-agnostic and would port to any SQL-compatible warehouse with minimal refactoring.

---

3. Current System Status

3.1 Data Coverage

Category	Records	Coverage
Companies tracked	504	S&P 500 via iShares IVV ETF
Financial data points	174,977	12 quarters per ticker across 3 statement types
Calendarized financials	176,045	100% of tracked tickers
Consensus estimates (raw)	11,456	Revenue + EPS, annual periods
Calendarized estimates	40,208	Seasonal split to calendar quarters
Earnings transcripts	132	21 tickers (4% — FMP Starter plan limitation)
Research data records	523	Web search-sourced qualitative data
Capital structure snapshots	~504	One per ticker, refreshed weekly
Revenue segments	~504	Product + geographic breakdown

3.2 FMP Starter Plan Realities

The system operates on a Financial Modeling Prep (FMP) Starter plan, which provides strong coverage for core financial data but has known limitations.

Strong coverage (>95%): - Quarterly and annual financial statements (income, balance sheet, cash flow) - Annual consensus estimates (Revenue and EPS, 10 years forward) - Capital structure and valuation metrics (beta, WACC, EV, market cap) - S&P 500 constituent holdings via IVV ETF - Company profiles and sector/industry classification

Known gaps (Starter plan limitations): - **Quarterly consensus estimates**: Not available. Annual estimates are calendarized using seasonal decomposition from historical actuals. - **Earnings surprises (actuals)**: Not available as a separate endpoint. Actuals are derived from financial statement data. - **Earnings transcripts**: Limited to approximately 4% coverage (132 transcripts across 21 tickers). Full transcript access requires a higher-tier plan. - **Intraday pricing data**: Not utilised. The system focuses on fundamental analysis rather than technical trading signals.

Mitigation strategies: - Seasonal decomposition bridges the quarterly estimate gap with company-specific historical patterns - Web search-based research data collection supplements gaps in structured

data - Cross-source validation (Layer 2) detects inconsistencies arising from data source limitations - All comparisons between estimates and actuals carry explicit basis warnings

3.3 Automated Update Cadence

Operation	Frequency	Last Run	Result
Full data refresh	Weekly (Sunday)	2026-03-01 22:00 UTC	504 tickers, 28,067 records, 0 failures
Incremental update	Daily (weekdays)	2026-03-02 00:30 UTC	35 tickers, 10,212 records
Data quality audit	Daily	2026-03-02	Score: 67/100, 22/35 checks passed
System valuation	Daily	2026-03-02	Full report with USD range and audit scores

3.4 Future-Proofing

The database schema includes nullable columns for international expansion: - `reporting_currency` (varchar 3) — for multi-currency support - `usd_fx_rate` (real) — for foreign exchange conversion tracking - `accounting_standard` (varchar 10) — for IFRS vs US GAAP distinction - `reporting_language` (varchar 5) — for multilingual filing support - `primary_exchange` (varchar 10) — for multi-exchange coverage

These columns are currently unused (all S&P 500 companies report in USD under US GAAP) but eliminate the need for schema migration when expanding to international markets.

3.5 Observability & Workflow Transformation

Built-In Observability Framework Nexus treats observability as a first-class concern, not an afterthought. Two daily automated processes provide continuous system health monitoring:

- **Daily system valuation** (Agent 4, 6:00 PM UTC): Gathers metrics from every database table — record counts, coverage percentages, citation rates, last-update timestamps — and submits them to GPT-5-mini configured as a senior fintech valuation analyst. The output is a structured report with fair value range (low/base/high USD), audit scores across four dimensions (architecture, data quality, analytical quality, compliance readiness), an overall letter grade, and prioritised action items. All reports are persisted to `system_valuation_reports` for trend analysis.
- **Daily data quality audit** (Agent 4, immediately after valuation): Runs the full 35-check audit framework, produces per-ticker scores, identifies critical flags requiring action, and triggers LLM-assisted anomaly classification for the worst-scoring tickers. Reports are persisted to `data_quality_reports` with full scoring detail.

Both processes produce structured, machine-readable output alongside human-readable summaries — enabling future integration with dashboards, alerting systems, or compliance reporting tools.

Real-Time Progress Broadcasting All scheduled operations broadcast real-time progress to linked Telegram users:

- **Weekly full update** (504 tickers): Progress updates every 20% completion, with final summary showing tickers processed, records inserted, failures, and duration
- **Daily incremental update** (35+ tickers): Progress updates every 25% completion, with breakdown of recent reporters vs random sample
- **Agent 4 pipelines**: Start notification, intermediate status, and full report delivery for both valuation and data quality runs
- **Error escalation**: Failures in any scheduled operation immediately broadcast an error summary with the first 100 characters of the error message

Transforming Investment Workflows The system demonstrates how complex, traditionally manual investment workflows can be restructured as auditable, automated AI processes:

Traditional Workflow	Nexus Transformation
Analyst manually pulls quarterly financials from SEC filings	Agent 1 auto-collects 28 metrics across 3 statement types for 504 tickers with full citation tracking
Research associate normalises data in Excel for cross-company comparison	Calendarization engine applies day-weighted proration automatically, eliminating FYE distortion
Senior analyst spot-checks data quality by reviewing outliers	35 automated checks run daily with severity-weighted scoring and LLM-assisted anomaly triage
Portfolio manager requests ad-hoc analysis via email	Telegram bot delivers instant multi-factor analysis with DCF, factor positioning, and trade setup
Compliance team reviews research for tone and disclosure	Agent 4 continuously benchmarks against institutional standards with automated tone enforcement

Each transformation preserves the analytical rigour of the manual process while eliminating latency, human error, and capacity constraints. The audit trail — citation tracking, collection run logs, quality reports — provides the same (or greater) accountability that regulators and institutional clients expect.

---

4. Technology Stack

Component	Technology
Frontend	React 18, TypeScript, Tailwind CSS, shadcn/ui, Framer Motion
Backend	Node.js, Express, TypeScript
Database	PostgreSQL with Drizzle ORM
AI / LLM	OpenAI GPT-5-mini
Vector Search	pgvector 0.8.0 (HNSW index, cosine similarity)

Component	Technology
Embeddings	OpenAI text-embedding-3-small (1536 dimensions)
Authentication	Replit Auth (OpenID Connect) via Passport.js
Messaging	Telegram Bot API with long-polling
Data Sources	FMP API, iShares IVV ETF, DuckDuckGo/Bing web search
Hosting	Replit with PostgreSQL

---

5. Compliance & Institutional Standards

- **Full audit trail**: Every data point traceable to source with URL, date, and quote text
- **Deterministic reproducibility**: Layer 1 and Layer 2 checks produce identical results on identical data
- **Conservative methodology**: DCF models use exponential fade to terminal growth; no extrapolation beyond supported horizons
- **Explicit uncertainty disclosure**: All estimate-to-actual comparisons carry basis warnings; coverage gaps are surfaced rather than hidden
- **Professional tone enforcement**: Agent 4 continuously benchmarks output against WSJ, FT, Bloomberg, and Reuters standards
- **No silent fallbacks**: Missing data is flagged explicitly rather than substituted with defaults

5.1 Future-Proof Architecture: Path to LangGraph Migration

Why Custom 4-Agent Architecture Was Chosen The decision to build a custom agent framework rather than adopting an existing one (LangChain, CrewAI, AutoGen) was driven by four factors:

1. **Speed of development**: Direct function calls between agents eliminated the learning curve and abstraction overhead of a framework, allowing the full system to be built in under two weeks
2. **Predictable costs**: Agent communication via database reads and function calls avoids the token overhead of framework-mediated inter-agent conversations, keeping per-run costs consistent and forecastable
3. **Deterministic control**: The data quality audit (35 checks) and calendarization engine require exact reproducibility — framework-managed state and routing introduce unnecessary variability
4. **Stability**: No dependency on rapidly evolving framework APIs; the custom architecture uses only stable Node.js, PostgreSQL, and OpenAI SDK interfaces

When LangGraph Would Make Sense LangGraph migration becomes justified when the system evolves to require:

- **Dynamic routing**: Conditional agent activation based on runtime data (e.g., "if earnings surprise > 20%, trigger deep-dive analysis automatically")
- **Agent-to-agent negotiation**: Scenarios where Agent 2 (Analyst) needs to request specific data from Agent 1 (Collector) based on analysis gaps, with multi-turn back-and-forth

- **Complex branching**: 8+ agent nodes with parallel fan-out/fan-in patterns, retry logic, and conditional error recovery paths
- **Multi-tenant orchestration**: Serving multiple clients with different agent configurations, data permissions, and output formats

High-Level Migration Plan

```
LangGraph: NexusResearchPipeline

Nodes:
  universe_update     → Agent 1: Refresh S&P 500 holdings from IVV ETF
  data_collection     → Agent 1: Run 5 sub-collectors + research for each ticker
  calendarization     → Agent 1: Day-weighted proration to calendar quarters
  data_quality_audit  → Agent 4: 35-check audit + LLM anomaly classification
  analysis            → Agent 2: Multi-factor synthesis, Nexus Score, DCF
  polishing           → Agent 4: Institutional tone enforcement, competitor benchmarking
  delivery            → Agent 3: Telegram broadcast, web dashboard update

Edges:
  universe_update  → data_collection
  data_collection  → calendarization
  calendarization  → data_quality_audit
  data_quality_audit → analysis  (conditional: proceed only if score > 50)
  analysis         → polishing
  polishing        → delivery

Orchestrator (Agent 3):
  Manages cron-triggered graph executions, progress callbacks,
  and Telegram notifications at each node transition.

State:
  ticker_list, financial_data, quality_score, analysis_output,
  polish_context - persisted in PostgreSQL between runs.
```

Benefits at Enterprise Scale

- **Built-in observability**: LangGraph provides execution traces, node-level timing, and state snapshots — replacing custom logging
- **State management**: Automatic checkpointing enables pipeline resumption from the last successful node after failures
- **Compliance guardrails**: Framework-level input/output validation at each node boundary, auditable by default
- **Debugging**: Visual graph editor shows exact execution path, making production issues faster to diagnose

Current Status The custom 4-agent architecture is production-ready today, processing 504 tickers across 176,000+ calendarized records with 35 automated quality checks. LangGraph migration is planned for when dynamic agent-to-agent workflows or multi-tenant requirements emerge. The

migration path is well-defined and low-risk because the existing agent boundaries map cleanly to LangGraph nodes.

5.2 Product Roadmap & Stakeholder Collaboration

Short-Term (Next 3 Months)

- **Global coverage expansion**: Activate the future-proofing columns (`reporting_currency`, `usd_fx_rate`, `accounting_standard`) to onboard FTSE 100, Euro Stoxx 50, and Nikkei 225 constituents. The schema, calendarization engine, and audit framework are already international-ready — only the data ingestion layer requires FMP API tier upgrade and FX rate integration.
- **Backtesting migration**: Port the Python-based factor backtesting system (currently running as spawned scripts) into the TypeScript backend for unified execution, eliminating the Python runtime dependency and enabling real-time factor screen queries via the Telegram bot.
- **Enhanced transcript coverage**: Upgrade FMP plan tier to unlock full earnings transcript access (currently ~4% coverage), enabling comprehensive management tone analysis and AI-parsed guidance extraction across all 504 tickers.

Medium-Term (3–6 Months)

- **LangGraph migration**: Transition from custom scheduler-driven orchestration to LangGraph's graph-based execution model when dynamic routing requirements emerge (e.g., conditional deep-dive analysis triggered by earnings surprises, automated re-collection on data quality failures).
- **Multi-user analytics**: Extend the current single-user Telegram interface to support team-based research workflows — shared watchlists, collaborative annotations, and role-based access to analysis outputs.
- **Real-time data integration**: Add WebSocket-based live price feeds and intraday factor signal updates, complementing the current end-of-day batch processing model.

Long-Term (6–12 Months)

- **Institutional API layer**: REST/GraphQL API exposing calendarized financials, quality scores, and Nexus Score ratings for integration with portfolio management systems, order management platforms, and compliance dashboards.
- **Custom model fine-tuning**: Fine-tune a domain-specific LLM on the accumulated corpus of Nexus research outputs, quality reports, and competitive analysis — reducing dependence on general-purpose models and improving consistency of institutional tone.

Stakeholder Collaboration Model The architecture is designed to serve both fundamental and quantitative investment professionals:

- **For fundamental analysts**: Agent 2 delivers narrative-driven research with DCF models, management commentary analysis, and competitive positioning — the qualitative depth expected by bottom-up stock pickers
- **For quantitative researchers**: The calendarized financial database, factor positioning system, and cross-sectional quintile rankings provide the structured, normalised data that quant strategies require — accessible via direct database queries or API endpoints

- **For portfolio managers**: The Nexus Score (0–100 composite) and trade setup synthesis provide a standardised, comparable signal across the full S&P 500 universe, suitable for portfolio construction screening and risk overlay
- **For compliance and operations**: The 35-check data quality audit, full citation tracking, and deterministic reproducibility satisfy the documentation and auditability requirements of institutional compliance frameworks

This multi-stakeholder design reflects the collaborative nature of institutional investment teams, where research, portfolio construction, risk management, and compliance operate as interconnected functions sharing a common data foundation.

---

6. GitHub Strategy

6.1 Repository Setup

The recommended approach is a **private GitHub repository** to protect proprietary logic while demonstrating professional development practices.

What to include in the repository: - All source code (frontend, backend, shared models) - Schema definitions and migration configuration - This documentation file and the README - Configuration files (tsconfig, drizzle config, package.json)

What to exclude (add to `.gitignore`): - `.env` files containing API keys and database credentials - `node_modules/` directory - `macro_data/output/` (generated CSV/JSON data files) - Any exported financial data or proprietary research outputs - Database dumps or backups

Current Development State (MVP Phase) The working system is currently hosted on Replit with a PostgreSQL backend. All core functionality (data collection, calendarization, 35-check audit, DCF modelling, multi-agent orchestration) is live and running daily. A private GitHub repository has been created containing full architecture documentation and demo screenshots. Migration to a production environment (LangGraph orchestration + enterprise data platform) is the planned next phase for institutional deployment.

6.2 Making the Project Interview-Ready

Commit history: The project has a natural, organic commit history spanning from initial scaffolding (Feb 22) through progressive feature additions. This is more compelling than a single monolithic commit because it demonstrates iterative development discipline.

README structure for the repository root: 1. One-paragraph project summary (what it does, why it matters) 2. Architecture diagram (text-based or simple image showing the 4-agent flow) 3. Key metrics (504 tickers, 176k calendarized records, 35 automated checks) 4. Technology stack table 5. "How to run" section with environment variable requirements 6. Link to this full architecture document

What interviewers look for: - Clean separation of concerns (agents with defined boundaries) - Evidence of real data handling at scale (not toy datasets) - Automated quality assurance (the 3-layer audit framework) - Professional documentation (this document)

Showing progress without exposing code: Share this architecture document, the data quality report format (Section 2.8), and coverage statistics. These demonstrate engineering depth without revealing implementation details.

6.3 Protecting Proprietary Information

The following elements represent genuine intellectual property and should remain private: - The specific audit check thresholds and scoring formula - Agent 2's Nexus Score pillar weights and trade setup synthesis methodology - The calendarization day-weighting algorithm implementation - Terminal growth rate fade function parameters - Agent 4's polishing and tone enforcement prompts

For interview discussions, describe the *approach* (e.g., "we use day-weighted proration for calendarization") without sharing the specific implementation.

---

7. Development Log

A chronological record of key milestones from project inception to current state.

Week 1 — Foundation (Feb 22–23, 2026)

Day 1 (Feb 22): Project scaffolded on Replit with React/TypeScript frontend and Node.js/Express backend. PostgreSQL database provisioned. Initial 3-agent architecture established: Agent 1 (knowledge base and data ingestion), Agent 2 (analysis and topic generation), Agent 3 (scheduler). Telegram bot integrated for research delivery. Python data pipeline scripts added for macro indicators, S&P 500 history, and factor backtesting. Daily topic generation and morning briefings implemented. RSS news feed ingestion from major financial sources. First working prototype of AI-driven research topic suggestions.

Day 2 (Feb 23): Expanded data pipeline with company financials, valuations, and earnings revisions. Added Yahoo Finance and FMP API integration. Built the factor screener with multiple screening modes. Chart rendering via QuickChart.io for Telegram delivery. Trade setup logic implemented with multi-factor weighted analysis.

Week 1 — Analysis Engine (Feb 24–28, 2026)

Key decisions made: - Chose FMP API as primary data source (comprehensive coverage, reasonable cost) - Selected iShares IVV ETF as the S&P 500 universe source (daily updated holdings, no licensing issues) - Adopted conservative DCF methodology with exponential fade to terminal growth

Major features added: - DCF modelling with segment-weighted terminal growth rates - Nexus Score: 0–100 composite rating across five investment pillars - Vector RAG system using pgvector for semantic search across news and transcripts - Agent 4 (Research & Polishing Intelligence) added — competitive analysis of WSJ/FT/Bloomberg/Reuters standards - Automated data collection orchestrator with batch processing (groups of 5, Promise.allSettled for fault isolation) - Universe manager tracking S&P 500 composition via IVV ETF holdings - Web search wrapper with DuckDuckGo + Bing fallback for market research - Research data collector for qualitative insights (growth prospects, ownership, pricing trends)

Week 2 — Institutional Quality (Mar 1–2, 2026)

Architecture evolution: Shifted from Python-heavy data pipelines to a unified TypeScript data collection framework. Five specialised sub-collectors replaced the monolithic Python scripts: consensus estimates, financials, capital structure, transcripts, and segments.

Major features added: - Daily valuation and audit system (Agent 4) — system-wide health assessment with fair value estimates - Data quality audit framework: 30 deterministic checks across 6 categories - Calendarization engine: day-weighted proration for all 504 tickers, eliminating FYE distortion - Calendarized estimates: annual consensus split to calendar quarters using seasonal weights from actuals - Forward estimates API: LTM/NTM/NTM+1 with explicit basis warnings - Cross-source validation (Layer 2): 5 checks comparing data across independent sources - LLM-assisted audit (Layer 3): GPT-5-mini reviews worst-scoring flagged tickers - Future-proofing schema columns for international expansion (currency, FX rate, accounting standard, exchange) - Full professional documentation (this document)

Key numbers at completion: - 504 companies tracked (full S&P 500) - 174,977 raw financial records - 176,045 calendarized financial records - 40,208 calendarized estimate records - 35 automated data quality checks - Weekly full update: 504 tickers in ~3 hours, 0 failures - Daily incremental update: 35+ tickers in ~16 minutes

---

8. Total Cost Summary

8.1 Infrastructure Costs

Service	Monthly Cost	Notes
Replit Pro	$25/month	Hosting, PostgreSQL database, CI/CD
FMP API (Starter)	$30/month	Financial data, estimates, profiles
OpenAI API	~$5–10/month	GPT-5-mini for analysis, audit, polishing
Telegram Bot API	Free	Message delivery and bot hosting
Total monthly	**~$60–65/month**	

8.2 Cumulative Costs (Feb 22 – Mar 2, 2026)

Category	Estimated Cost
Replit (partial month)	~$12
FMP API (partial month)	~$15
OpenAI API usage	~$8
Total to date	**~$35**

8.3 Cost Context

For perspective, a single Bloomberg Terminal costs $24,000/year ($2,000/month). A FactSet subscription for a small fund starts at approximately $12,000/year. Nexus delivers comparable analytical capabilities for the S&P 500 universe at approximately 0.4% of Bloomberg Terminal cost.

The primary ongoing costs scale with: - **FMP API calls**: ~5,000 calls/week for full updates (within Starter plan limits) - **OpenAI tokens**: ~50,000 tokens/day for analysis, audit, and polishing (well within standard usage) - **Database storage**: ~500MB for all financial data, growing approximately 30MB/month

---

9. Agentic Framework Comparison — Interview Preparation

9.1 Framework Overview

The AI industry has developed several frameworks for building multi-agent systems. Understanding these is valuable for technical discussions, even if the project uses a custom architecture.

LangChain **What it does**: LangChain is a toolkit for building applications powered by large language models. It provides pre-built components for connecting LLMs to databases, APIs, documents, and other tools. Think of it as a Swiss Army knife for LLM applications.

Strengths: Large ecosystem, extensive documentation, works with many LLM providers. Good for simple chain-of-thought workflows (e.g., "search the web, then summarise, then answer").

Limitations: Can become overly complex for production systems. The abstraction layers sometimes make debugging difficult. Not specifically designed for multi-agent coordination.

Compared to Nexus: LangChain would handle individual LLM calls (like our Agent 2 analysis or Agent 4 polishing) but would not naturally enforce the strict agent boundaries we have (Agent 1 writes data, Agent 2 only reads, etc.).

AutoGen (Microsoft) **What it does**: AutoGen enables conversations between multiple AI agents. Agents can talk to each other, delegate tasks, and collaborate on complex problems. It was designed for scenarios where multiple AI perspectives improve the outcome.

Strengths: Good for research and brainstorming workflows where agents debate or iterate. Built-in support for human-in-the-loop approval.

Limitations: Designed for conversational multi-agent patterns, which can lead to unpredictable token usage. Less suited for deterministic data pipelines where you need consistent, repeatable results.

Compared to Nexus: AutoGen's conversational approach would be wasteful for our use case. Our agents do not need to "discuss" — Agent 1 collects data, Agent 2 analyses it. The pipeline is sequential and deterministic, not conversational.

CrewAI **What it does**: CrewAI organises AI agents into "crews" with defined roles, goals, and tools. Each agent has a specific job title and responsibility, similar to how a human team operates.

Strengths: Intuitive role-based design. Easy to understand and explain. Good for workflows where each agent has a distinct specialty.

Limitations: Relatively new framework with a smaller ecosystem. Can be heavy on token usage because agents explain their reasoning to each other. Less control over execution order compared to custom solutions.

Compared to Nexus: CrewAI's role-based model is the closest conceptual match to our 4-agent architecture. However, our implementation is more efficient because we use direct function calls rather than LLM-mediated communication between agents.

Semantic Kernel (Microsoft) **What it does**: Semantic Kernel is Microsoft's SDK for integrating AI into enterprise applications. It focuses on "plugins" — reusable AI capabilities that can be composed into larger workflows.

Strengths: Strong enterprise integration (Azure, Microsoft 365). Good for adding AI capabilities to existing business applications. Type-safe, well-structured.

Limitations: Heavily tied to the Microsoft ecosystem. More of an SDK than an agent framework — less focus on autonomous multi-agent behaviour.

Compared to Nexus: Semantic Kernel would be relevant if we were building within a Microsoft enterprise environment. Our independent architecture gives us more flexibility in choosing tools and providers.

Retrieval-Augmented Generation (RAG) in Nexus Nexus implements a hybrid RAG architecture that combines vector-based semantic search with structured deterministic data to produce context-rich, grounded analysis.

Vector search layer: pgvector 0.8.0 with HNSW indexing (cosine similarity) stores 1536-dimensional embeddings generated by OpenAI text-embedding-3-small. News articles (title + summary) and earnings transcript chunks (~500 tokens each) are embedded and indexed for semantic retrieval. The system auto-embeds new content after scheduled RSS fetches and transcript collection runs.

Hybrid context injection: When Agent 2 generates analysis, the context window receives both: - **Deterministic structured data**: Calendarized financials, consensus estimates, DCF model outputs, capital structure metrics, factor positioning quintiles — pulled directly from PostgreSQL with exact values - **Semantic vector context**: Relevant news articles, transcript excerpts, and research data retrieved via cosine similarity search against the user's query — providing narrative context, management commentary, and market sentiment that structured data cannot capture

This hybrid approach ensures that analysis is grounded in auditable financial data while enriched with the qualitative context that institutional investors expect. The vector RAG layer is controlled by the `ENABLE_VECTOR_RAG` feature flag and is purely additive — disabling it produces analysis based solely on structured data, with no degradation in core functionality.

Compared to pure RAG systems: Many AI research tools rely entirely on retrieval-augmented generation, which can produce plausible-sounding but ungrounded analysis. Nexus inverts this pattern: the foundation is always deterministic financial data with full citation tracking, and RAG provides supplementary context rather than primary evidence.

LangGraph **What it does**: LangGraph extends LangChain with a graph-based orchestration model. Agents and tasks are represented as nodes in a directed graph, with edges defining the flow of data and control. This gives precise control over execution order, branching, and error handling.

Strengths: Best-in-class orchestration control. Supports cycles (agent A can call agent B, which can call agent A again if needed). Built-in state management and checkpointing. Production-ready.

Limitations: Steeper learning curve than simpler frameworks. Requires understanding graph theory concepts. Tied to the LangChain ecosystem.

Compared to Nexus: LangGraph is the most architecturally similar framework to what we have built. Our custom scheduler (Agent 3) essentially implements a graph-based execution model with cron-triggered nodes.

9.2 How Nexus Would Look in LangGraph

If we migrated the Nexus architecture to LangGraph, the conceptual mapping would be:

```
Graph: NexusResearchPipeline

Nodes:
  - universe_update    (Agent 1: refresh S&P 500 holdings)
  - data_collection    (Agent 1: collect financials for each ticker)
  - calendarization    (Agent 1: normalise to calendar quarters)
  - data_quality_audit (Agent 4: run 35 checks + LLM review)
  - analysis           (Agent 2: generate research output)
  - polishing          (Agent 4: apply institutional tone standards)
  - delivery           (Agent 3: broadcast via Telegram)

Edges:
  universe_update  --> data_collection
  data_collection  --> calendarization
  calendarization  --> data_quality_audit
  data_quality_audit --> analysis (conditional: only if score > 50)
  analysis         --> polishing
  polishing        --> delivery

State:
  - ticker_list: string[]
  - financial_data: Record<string, FinancialRecord[]>
  - quality_score: number
  - analysis_output: string
```

The key advantage of LangGraph would be built-in checkpointing (if the pipeline fails at calendarization, it can resume from the last successful node) and visual debugging tools. The key disadvantage is additional framework dependency and the overhead of serialising our database-heavy state into LangGraph's state management.

9.3 Sample Interview Answers

Q: "What agentic frameworks have you worked with?"

"I built Nexus using a custom 4-agent architecture in TypeScript because the standard frameworks — LangChain, AutoGen, CrewAI — introduce overhead that did not match our use case. Our agents have strict role boundaries: Agent 1 handles all data collection, Agent 2 does analysis, Agent 3 manages scheduling, and Agent 4 handles quality assurance. The communication is direct function calls and database reads, not LLM-mediated conversations, which keeps costs predictable and results deterministic. If I were to migrate to a framework, LangGraph would be the closest fit because it offers graph-based orchestration similar to what we already have."

Q: "Why didn't you use LangChain or CrewAI?"

"I evaluated them early on and decided the trade-offs did not make sense for this project. LangChain is excellent for simple LLM chains, but our pipeline is more about data engineering than LLM conversations — we are processing 175,000 financial records and running 35 deterministic quality checks. CrewAI's role-based model is conceptually similar to our architecture, but it adds token overhead because agents explain their reasoning to each other. Our agents communicate through the database and direct function calls, which is faster and cheaper."

Q: "How do you handle agent coordination?"

"Through a combination of scheduled cron jobs and event-driven triggers. Agent 3, the scheduler, manages the timing — weekly full updates on Sunday, daily incremental updates on weekdays, daily quality audits in the evening. The agents are decoupled by design: Agent 1 writes to the database, Agent 2 reads from it. There is no bidirectional communication that could create deadlocks or race conditions. If Agent 2 discovers missing data during analysis, it sends a single collection request back to Agent 1 and waits."

Q: "How would you scale this system?"

"The architecture is already designed for horizontal scaling. The batch processing uses Promise.allSettled with groups of 5 tickers, so increasing concurrency is a configuration change. The database queries use DISTINCT ON patterns optimised for per-ticker lookups. For true enterprise scale — say, covering 10,000 global equities — I would move the data collection to a job queue (like BullMQ or AWS SQS), add a caching layer for frequently accessed data, and potentially split Agent 1 into separate microservices for each data source."

---

10. Full Replication Guide

The following prompt sequence rebuilds the entire Nexus system from scratch on a fresh Replit project. Each step is self-contained and should be executed in order. Wait for each step to complete before proceeding to the next.

Prerequisites

- Replit account with Pro subscription
- FMP API key (Starter plan or above)
- OpenAI API key

- Telegram Bot Token (from @BotFather)

Step 1: Project Scaffolding and Core Architecture

```
Create an AI investment research platform called "Nexus" with a 4-agent architecture:

Agent 1 (Data Collector): Handles all external data acquisition and database writes
Agent 2 (Analyst): Reads data and generates investment analysis
Agent 3 (Scheduler): Manages cron-based automation
Agent 4 (Research Intelligence): Quality assurance and tone enforcement

Tech stack: React 18 + TypeScript frontend, Node.js/Express backend, PostgreSQL with Drizzle OI

Frontend: Dark theme, Tailwind CSS, shadcn/ui components, Wouter routing.
Backend: RESTful API with /api/ prefix.

Set up the database schema with these tables:
- users (id, username, telegramChatId)
- knowledge_base (id, userId, type, title, content, metadata, createdAt)
- ingestion_logs (id, scriptName, status, output, createdAt)

Add Replit Auth (OpenID Connect) via Passport.js.

Create a dashboard page showing system status, and a knowledge base page for data ingestion.
```

Step 2: Data Pipeline and Macro Indicators

```
Add Python data pipeline scripts for macro economic data:

1. fetch_macro_data.py: Pull from FRED API - yields (2Y, 10Y, 30Y), fed funds rate, CPI, PCE, (

2. fetch_sp500_history.py: Download S&P 500 index + 11 GICS sector ETF price history via Yahoo

3. compute_performance.py: Calculate 1d/5d/1m/3m/6m/1y returns for all tracked assets.

4. fetch_daily_factor_returns.py: Pull Fama-French factor returns and style quintile data.

Add a runPythonScript() helper that spawns Python processes and captures output. Register scrip

Add RSS news feed fetching from Reuters, Bloomberg, WSJ, FT, CNBC, MarketWatch. Store articles
```

Step 3: Telegram Bot and Agent 2 Analysis

```
Build a full-featured Telegram bot for Nexus:

Commands:
- /start: Link Telegram to Nexus account via unique code
- /topics: View pending research topics
- /generate: Generate 3 new AI-driven research topics based on current market context
```

```
- /analyze [ticker]: Full multi-factor analysis with trade setup
- /dcf [ticker]: DCF fair value model
- /wacc [ticker]: WACC calculation
- /health_check: System status

Agent 2 chat: Any natural language message triggers AI research response using:
- 90-day rolling news context
- Knowledge base entries
- Prior analyses (for consistency enforcement)
- Financial data context

Trade Setup: Multi-factor weighted synthesis - DCF fair value (25%), style/momentum (25%), earr

Nexus Score: 0-100 composite rating based on Factor Position, Financial Momentum, Valuation, Ea

Use OpenAI gpt-5-mini with max_completion_tokens (not temperature). Use cleanForTelegram() for
```

Step 4: FMP Data Collection Framework

```
Build Agent 1's data collection framework using Financial Modeling Prep (FMP) API:

Database tables:
- companies (ticker, name, sector, industry, isCurrentlyInSp500, fiscalYearEndMonth, fmpProfile
- company_financials (ticker, periodEnd, periodType, statementType, metricName, metricValue, cu
- consensus_estimates (ticker, asOfDate, period, periodType, metric, estimate, actual, numAnaly
- capital_structure (ticker, asOfDate, beta, marketCap, netDebt, sharesOutstanding, wacc, costO
- company_transcripts (ticker, fiscalYear, fiscalQuarter, transcriptDate, preparedRemarks, qaSe
- company_segments (ticker, periodEnd, segmentName, segmentType, revenue, operatingIncome, marg

Five sub-collectors in server/data-collectors.ts:
1. collectConsensusEstimates(ticker): Revenue + EPS annual estimates from FMP
2. collectFinancials(ticker): Income/BS/CF statements, last 12 quarters, 28 key metrics
3. collectCapitalStructure(ticker): Beta, WACC, EV, market cap, D/E
4. collectTranscripts(ticker): Earnings call text with AI sentiment parsing
5. collectSegments(ticker): Product + geographic revenue breakdown

Every record must include source_name, source_url, published_date, and quote_text for full cita

FMP client: Rate-limited wrapper with 300 calls/min limit. Auto-retry on 429 errors.

collectAllForTicker(ticker): Runs all 5 sub-collectors in parallel via Promise.all. Auto-create
```

Step 5: Universe Manager and Batch Orchestration

```
Build the S&P 500 universe manager and batch update orchestrator:

Universe Manager (server/universe-manager.ts):
- Fetch iShares IVV ETF holdings CSV to get current S&P 500 constituents
```

```
- Normalise tickers (BRK.B -> BRK-B, etc.)
- Detect additions, removals, and delistings
- Store in companies table with isCurrentlyInSp500 flag

Collector Orchestrator (server/collector-orchestrator.ts):
- runFullUpdate(): Process all ~503 tickers in batches of 5 using Promise.allSettled
- runIncrementalUpdate(): Target recent earnings reporters + random sample of 35 stale tickers
- Progress callback support for Telegram broadcasting
- Log each run to collection_runs table (runType, startedAt, completedAt, tickersProcessed, ti

Scheduler cron jobs:
- Sunday 10PM UTC: runFullUpdate() with Telegram progress every 20%
- Weekdays 00:30 UTC: runIncrementalUpdate() with Telegram progress every 25%
- Sunday 8PM UTC: Universe refresh
- Weekdays 00:00 UTC: Earnings calendar update and auto-refresh

All scheduled runs broadcast start, progress, and completion summary to all linked Telegram use
```

Step 6: Web Search and Research Data Collection

```
Add web search capability and research data collection:

Web Search Wrapper (server/web-search-wrapper.ts):
- Primary: DuckDuckGo HTML scraping
- Fallback: Bing search
- Extract CAGR sentences from market research pages
- FMP fallback: Calculate industry-average revenue CAGR from representative tickers
- 12-second timeout, user-agent spoofing

Research Data Collector (in server/data-collectors.ts):
- New table: company_research_data (ticker, category, dataKey, dataValue, numericValue, period
- Categories: growth_prospects, industry_metrics, guidance, ownership, management, pricing
- 6 search queries per ticker, max 4 executed, 1.5s delay between searches
- Skip if ticker refreshed within 7 days
- Integrate into collectAllForTicker() wrapped in try/catch (non-fatal)

Industry Research (server/industry-research.ts):
- Research long-term growth trends for any sector/industry
- Store in industry_trends_data table with full citations
- Auto-triggers from terminal growth research when no data exists
```

Step 7: DCF and Terminal Growth Modelling

```
Build the DCF fair value modelling system:

Terminal Growth Research:
- Primary method: Segment-weighted approach
  - Fetch FMP revenue segmentation for each ticker
```

```
  - Map each segment to industry CAGR from industry_trends_data
  - Compute revenue-weighted terminal growth rate
  - Apply exponential fade with industry-profile-specific parameters (secular high-growth: k=0
- Fallback: Blended consensus/historical/industry weighting
- Sensitivity table covering 4%-9% terminal growth range

DCF Model:
- Live Excel-compatible output with editable input cells
- All calculations use real formulas referencing input cells
- Components: revenue forecast, margin assumptions, WACC, terminal value, enterprise-to-equity
- Output: Low/Base/High fair value range in USD per share

Integrate into Agent 2's trade setup - DCF fair value gets 25% weight in the multi-factor synth
```

Step 8: Vector RAG System

```
Add hybrid vector search for context-enriched analysis:

Schema (shared/models/embeddings.ts):
- document_embeddings table with pgvector extension
- HNSW index for cosine similarity search
- 1536 dimensions (OpenAI text-embedding-3-small)

Vector RAG (server/vector-rag.ts):
- Embed news articles (title + summary) and transcript chunks (~500 tokens each)
- Feature flag: ENABLE_VECTOR_RAG env var
- Purely additive - vectorContext injected alongside existing structured data in Agent 2's syst
- Auto-embed new content after scheduled RSS fetch and transcript collection

Telegram commands: /rag (status), /rag embed (embed all existing data)
```

Step 9: Agent 4 — Research Polishing and Daily Valuation

```
Build Agent 4's two main functions:

Research Polishing (server/research-polish.ts):
- Schedule: Twice weekly (Tue+Sat 5PM UTC)
- Pipeline: analyzeCompetitors() -> analyzeTradFiCommentary() -> scanForumTheses() -> generatel
- Analyses competitor AI tools (Morningstar, FactSet, Bloomberg)
- Benchmarks against WSJ/FT/Bloomberg/Reuters standards
- Cooldown: 3s between web searches, max 10 per run
- Output: polishContext injected as "STYLE GUIDANCE" in Agent 2's system prompt
- Guardrails: "conservative, professional, institutional tone. Never adopt hype or retail-style
- Feature flag: ENABLE_RESEARCH_POLISH (set permanently to true)

Daily Valuation (server/daily-valuation.ts):
- Schedule: Daily 6PM UTC
- Gathers system metrics from all database tables
```

```
- Sends system snapshot to GPT-5-mini configured as senior fintech valuation analyst
- Output: Fair value range (low/base/high USD), audit scores (architecture, data quality, analy
- Table: system_valuation_reports
- Commands: /valuation, /valuation_run
```

Step 10: Calendarization Engine

```
Build the calendarization engine for fiscal-to-calendar quarter normalisation:

computeCalendarWeights(fqStartStr, fqEndStr):
- Accepts actual fiscal quarter start and end dates
- Computes day-weighted overlap with each calendar quarter (Jan-Mar, Apr-Jun, Jul-Sep, Oct-Dec)
- Quarter-aligned FYE (3,6,9,12): blend_method='direct', weight=1.0
- Non-aligned FYE: blend_method='prorated', day-weighted splits summing to 1.0
- Balance sheet: point-in-time (latest value per CQ, not prorated)

Table: calendarized_financials (ticker, calendarQuarter, calendarYear, calendarQ, statementType

Backfill: Process all company_financials records. Derive fqStart from previous period_end + 1 d

getCalendarizedTTM(ticker, asOfQuarter?): Sum last 4 calendar quarters of IS/CF metrics, only i

Estimates Calendarization:
- Table: calendarized_estimates (ticker, calendarQuarter, calendarYear, calendarQ, metric, esti
- Split annual consensus estimates to calendar quarters using seasonal weights from most recent
- All records carry basis_warning flagging non-GAAP vs GAAP mismatch

getForwardEstimates(ticker): Returns LTM/NTM/NTM+1 for revenue and EPS. LTM requires all 4 actu
```

Step 11: Data Quality Audit Framework

```
Build the 3-layer data quality audit framework in server/data-quality-audit.ts:

Layer 1 - 30 Deterministic Checks:
Category A (Freshness, 6 checks): data staleness, collection recency, source fetch lag, post-ea
Category B (Units, 6 checks): revenue magnitude jump, NI magnitude jump, revenue-to-NI ratio, r
Category C (Completeness, 3 checks): income statement (9 metrics), balance sheet (7 metrics), c
Category D (Sanity, 7 checks): gross margin, EPS cross-check, P/E range, beta range, D/E range
Category E (Fiscal, 5 checks): duplicates, period ordering, FYE alignment, TTM consistency, mis
Category F (Citations, 3 checks): citation completeness, segment weight sum, missing paired dat

Layer 2 - 5 Cross-Source Validation Checks (IDs 31-35):
- Estimate vs actual divergence (>50% in latest quarters)
- Calendarization coverage gap (>20 ticker gap)
- TTM revenue vs annual estimate (>200% gap)
- Capital structure vs NI/EPS implied shares (>50% gap)
- Seasonality weight consistency (>1% drift)
```

```
Layer 3 - LLM-Assisted Audit:
- Select up to 5 tickers with score < 90 and 2+ failed checks
- Gather financial context (20 records), capital structure (3 snapshots), estimates (10 records
- Ask GPT-5-mini to classify as actionable/monitor/benign with root cause and recommendation
- Daily rate limit: check data_quality_reports table, skip if Layer 3 already ran today
- Non-fatal: wrapped in try/catch, failures don't affect Layer 1/2

Scoring: (passed/total) x 90 + citation_bonus(5) + FYE_bonus(3) + ticker_avg_bonus(5)
Per-ticker: Start at 100, deduct by severity (critical -15, high -8, medium -4, low -2)

Table: data_quality_reports (reportDate, overallScore, checksPassed/Failed/Total, criticalFlags

Integrate into Agent 4's daily cron (6PM UTC) after valuation. Telegram commands: /data_quality
```

Step 12: Future-Proofing and Documentation

```
Add nullable future-proofing columns to the database schema:

companies table:
- reporting_currency varchar(3)
- accounting_standard varchar(10)
- reporting_language varchar(5)
- primary_exchange varchar(10)

company_financials table:
- reporting_currency varchar(3)
- usd_fx_rate real
- accounting_standard varchar(10)

All columns nullable, no existing code changes needed.

Run db:push to apply schema changes.

Generate comprehensive system documentation covering:
1. System overview and 4-agent architecture
2. Data quality framework (all 3 layers, 35 checks)
3. Calendarization methodology
4. Current coverage statistics
5. Technology stack and compliance standards
```

Document generated: March 2, 2026 System version: Nexus AI Investment Research Platform v1.0